Software Requirements Document

1. Product Perspective

The Fantasy Premier League Pro (FPL Pro) is a standalone web application designed to serve as a comprehensive simulation platform for football management. It integrates real-world football data into a virtual environment by connecting to the official Fantasy Premier League (FPL) API, which allows the system to retrieve real-life match statistics, player information, and game week results.

The software is developed as a technical project that emphasizes modular architecture, utilizing a Python-based backend and a React frontend. Beyond its functionality as a simulation tool, the system is specifically structured to demonstrate the practical application of multiple software design patterns, ensuring the architecture is maintainable, extensible, and follows clean code practices. While it manages core fantasy mechanics like team building and transfers, it operates independently of live match streaming or external payment systems.

2. Product Functions

The Fantasy Tactics League platform provides the following core functions:

- Team Management: Users can create and manage virtual football teams composed of real Premier League players. This includes building squads within a fixed virtual budget and adhering to specific squad size constraints.
- Transfer System: The system enables users to perform player transfers between match weeks. This allows for strategic squad improvements while remaining within the defined budget limits.
- Automated Scoring: The software automatically calculates weekly points based on the real-life performances and match statistics of the selected players in actual Premier League fixtures.
- Captain Selection: Users can designate a specific player as the team captain for each match week to earn bonus points (typically doubled) based on that player's performance.
- League Ranking and Standing: The system provides a leaderboard functionality that sorts and ranks users based on their accumulated weekly points, allowing for comparison and competition between participants.

3. User Characteristics

The expected users of the Fantasy Tactics League platform and their respective skill levels are categorized as follows:

- Football Fans: Individuals who actively follow the English Premier League and possess a basic understanding of football rules and match mechanics.
- Strategic Managers: Users who enjoy strategic simulations and are capable of performing data-driven decision-making, such as budget management and long-term squad planning.
- Academic Reviewers: Instructors and students who will evaluate the system's underlying architecture, focusing on the correct application of Software Design Patterns and modular code practices.

4. Constraints

The development and operation of the Fantasy Tactics League platform are subject to the following technical and project-related constraints:

- Budget Rules: Users are required to build and maintain their fantasy squads within a fixed virtual budget of £100.0m.
- Squad Composition: Teams must adhere to valid football formations, consisting of 11 players with specific positional requirements (e.g., 1 Goalkeeper, 3-5 Defenders, 3-5 Midfielders, and 1-3 Forwards).
- API Usage: To respect the rate limits of the external football-data.org API, the system must utilize data fetching scripts or cached data to ensure stability.
- Design Pattern Implementation: The software architecture must explicitly implement and document at least four distinct software design patterns, such as *Factory Method*, *Strategy*, *Observer*, or *Singleton*.
- Implementation Deadline: All project deliverables, including the functional software and documentation, must be finalized and submitted by early April 2026.
- Exclusions from Scope: The system will not support real-time match streaming, financial payment systems, mobile application platforms, or advanced AI-based prediction features.

5. System Features (Use Case Based)

Use Case Diagram

The system's functionality is centered on the interaction between the User (the fantasy manager) and the System, which integrates data from an external Football Data API. The diagram depicts the User performing core management tasks such as creating teams, transferring players, and selecting captains, while the System handles automated point calculations and ranking updates based on real-world match data.

Detailed Use Case Descriptions

Use Case 1: Create New Fantasy Team

- Title: Create New Fantasy Team
- Main Actor: User
- Goal: To select an initial squad of players within the defined budget and squad rules to participate in the league.
- Preconditions:
    1. The user has accessed the application.
    2. The user does not already have an active saved team.
    3. Real-world player data has been successfully fetched from the external API.
- Main Flow:
    1. User navigates to the "Team Creation" interface.
    2. System displays a list of available Premier League players with their costs and positions.
    3. User selects players to fill the required squad positions.
    4. System updates the "Remaining Budget" in real-time as players are added.
    5. User completes the squad and clicks "Save Team".
    6. System validates the squad against budget constraints and squad size rules.
    7. System saves the team to the database.

- Postconditions: A new fantasy team record is created, and the user is added to the league ranking system.
- Exceptions:
    - Budget Exceeded: If the total cost exceeds the fixed budget, the system prevents saving and displays an error message.
    - Invalid Squad Size: The system blocks saving if the user has not selected the full required number of players.

This process is illustrated in the following image 1.

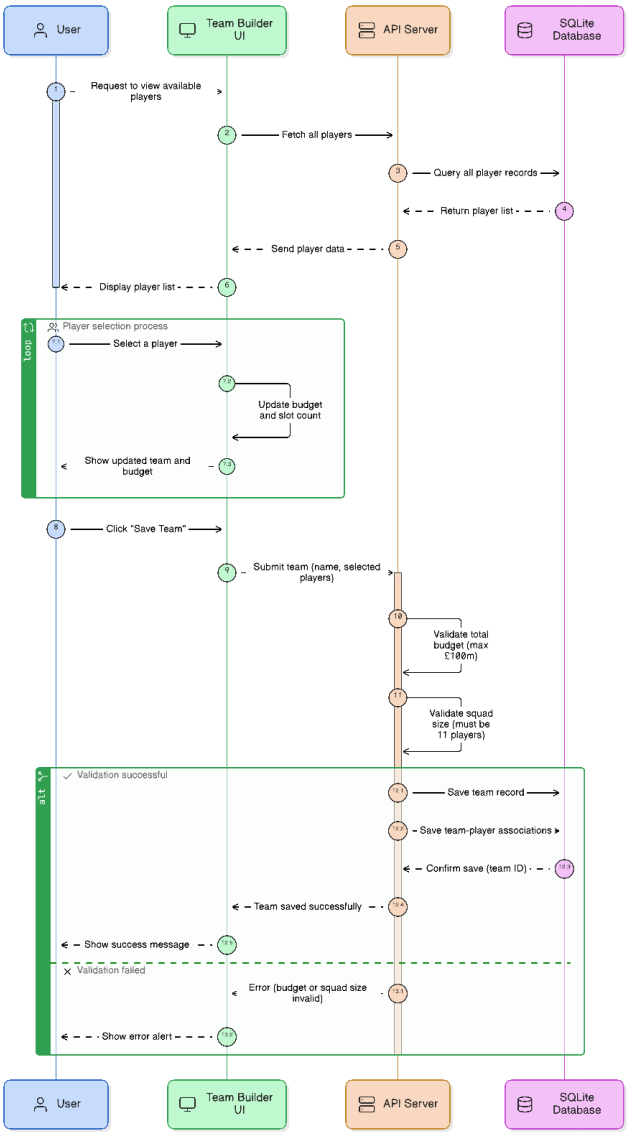

Image 1

Use Case 2: Player Transfer

- Title: Player Transfer Management
- Main Actor: User
- Goal: To replace an existing player in the squad with a new player while maintaining budget compliance.
- Preconditions:
    1. The user has an existing saved team.
    2. The weekly transfer window is open (match deadline has not passed).
- Main Flow:
    1. User views their current squad.
    2. User selects a player to remove (Out).
    3. System adds the value of the removed player back to the available budget.
    4. User searches for and selects a replacement player (In).
    5. System validates the new total squad cost against the budget.
    6. User confirms and saves the transfer.
- Postconditions: The squad list is updated in the database, and the user's available budget is adjusted.
- Exceptions:
    - Insufficient Funds: If the replacement player's cost exceeds the available budget, the transfer is rejected.

This process is illustrated in the following image 2.

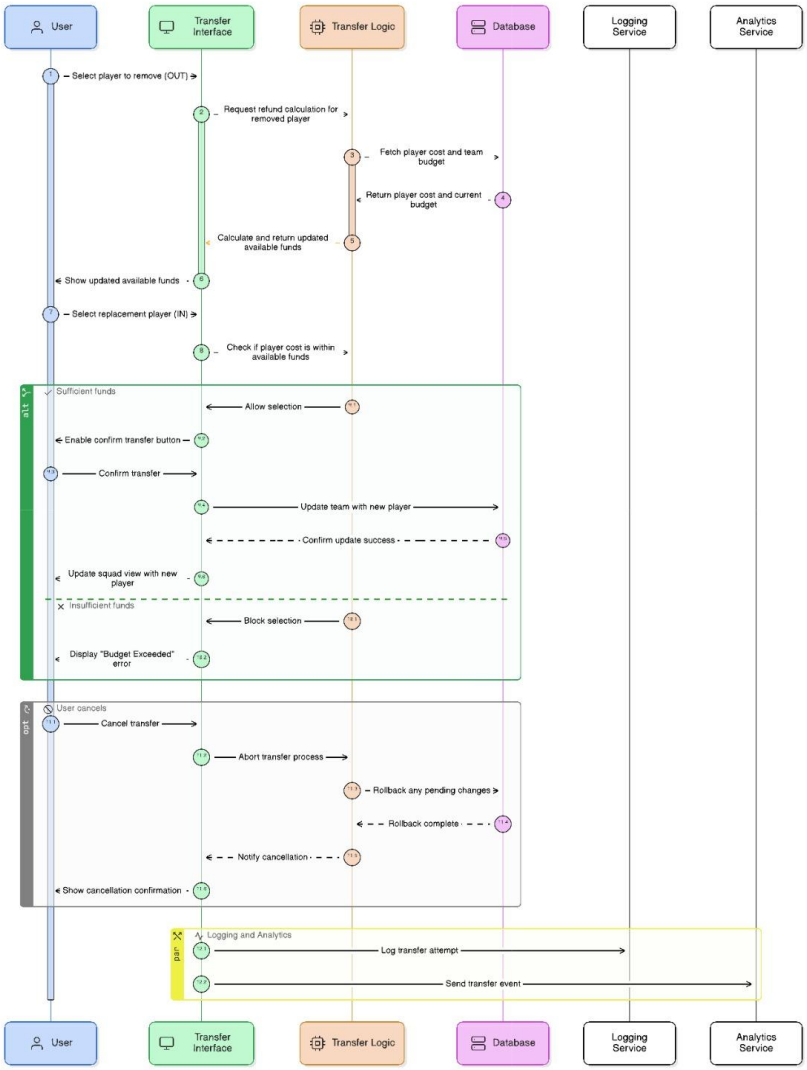

Image 2

Use Case 3: Captain Selection

- Title: Select Team Captain
- Main Actor: User
- Goal: To designate a player whose weekly points will be multiplied for a bonus.
- Preconditions: The user has a valid and complete squad of players.
- Main Flow:
    1. User opens the team management dashboard.
    2. System indicates the current active captain.
    3. User selects a different player from the squad list.
    4. User assigns the "Captain" role to the selected player.

5. System saves the change and updates the captain flag.

- Postconditions: The selected player is marked for bonus point calculation (e.g., double points) during the next scoring update.
- Exceptions: None.

This process is illustrated in the following image 3.

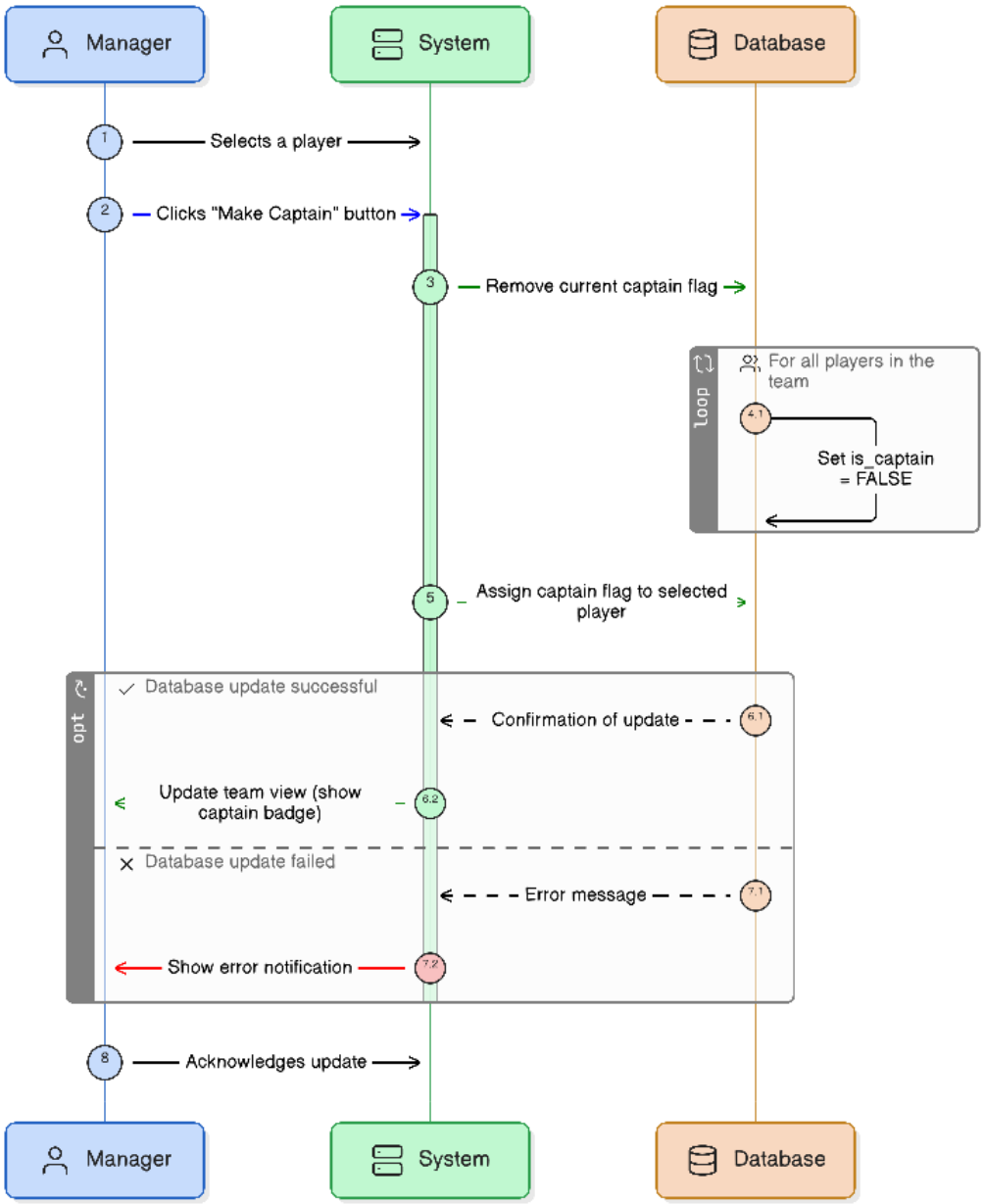

Image 3

Use Case 4: Viewing League Rankings

- Title: View Leaderboard
- Main Actor: User
- Goal: To view the current rankings and compare team performance against other users based on total points.
- Preconditions: The system has completed at least one weekly point calculation using real match data from the API.
- Main Flow:
    1. User navigates to the "League Table" or "Ranking" screen.
    2. System retrieves all user teams and their respective total points from the database.
    3. System sorts the teams in descending order of points.
    4. System displays the leaderboard showing Rank, Team Name, and Total Points.
- Postconditions: The user is presented with the most recent standings of the league.
- Exceptions: None.

This process is illustrated in the following image 4.

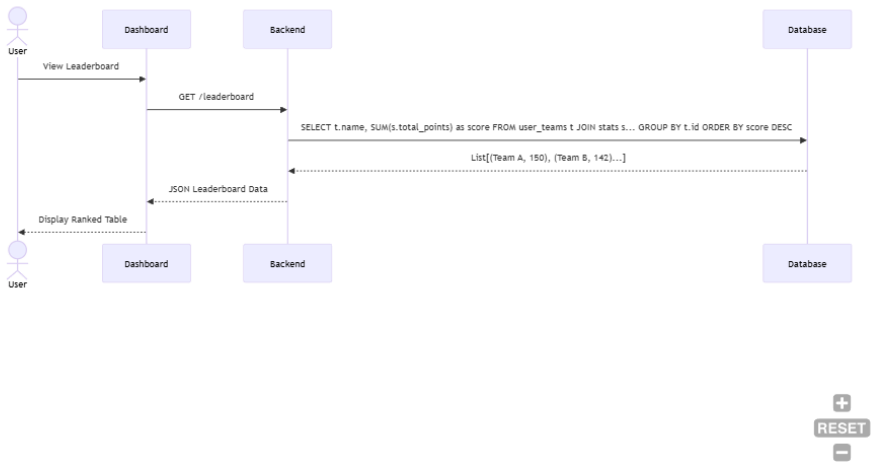

Image 4

6. Non-Functional Requirements

This section describes the system qualities and constraints that ensure the platform operates effectively beyond its core functionalities.

6.1 Usability

- Intuitive Interface: The "Team Builder" and dashboard must be designed so that football fans can manage teams and transfers without technical difficulty.
- Visual Feedback: The system must provide immediate visual feedback, such as highlighting negative budget balances in red, to help users avoid input errors.
- Efficient Selection: The player search and filtering tools (by name, team, or position) should allow users to quickly find and select players.

6.2 Performance

- API Efficiency: The response time for loading the player list (containing 800+ items) from the API must be under 1 second to ensure a smooth user experience.
- Processing Speed: Weekly point calculations for all users should be completed within seconds after fetching the latest match data.
- Data Flow: The integration with the external football-data.org API must be optimized to ensure consistent data flow without lagging the application.

6.3 Portability

- Cross-Platform Support: The system, built with a Python backend and React frontend, must be deployable on Windows, Linux, and macOS.
- Browser Compatibility: The application must function consistently across all modern web browsers.

6.4 Reliability

- Deterministic Scoring: The scoring logic must be stable and consistent; running the calculation for the same match data must always produce the same points.
- Failure Management: The system must handle API failures gracefully, such as by displaying cached or mock data if the live API service is unavailable.
- Architectural Integrity: The use of modular design patterns must ensure that the system remains maintainable and extensible throughout its lifecycle.

7. Software Interfaces

The Fantasy Tactics League application interacts with various external software components and systems to ensure data persistence and real-time updates:

- Football Data API: The system serves as a client to the football-data.org API to fetch Premier League-specific data, including player metadata, match results, and fixtures. This integration is critical for the automated scoring and ranking functions.
- External Data Format: All information retrieved from the external API is consumed and processed in JSON format to be mapped into the system's internal objects.
- Database Management System: A local database (such as SQLite) is utilized to store persistent application data, including user squad configurations, historical point totals, and player price histories.
- Version Control: GitHub is used as the primary software interface for collaborative development, source code versioning, and tracking team contributions through issues and pull requests.
- Architectural Documentation Tools: The system's design patterns and UML diagrams are developed and maintained using specialized software interfaces such as Draw.io or Lucidchart.